

Samples

Samples are the most common way to make sound with tidal and strudel. A sample is a (commonly short) piece of audio that is used as a basis for sound generation, undergoing various transformations. Music that is based on samples can be thought of as a collage of sound. [Read more about Sampling](#)

Strudel allows loading samples in the form of audio files of various formats (wav, mp3, ogg) from any publicly available URL.

Default Samples

By default, strudel comes with a built-in "sample map", providing a solid base to play with.

```
1 s("bd sd [~ bd] sd,hh*16, misc")
```

Here, we are using the `s` function to play back different default samples (`bd`, `sd`, `hh` and `misc`) to get a drum beat.

For drum sounds, strudel uses the comprehensive [tidal-drum-machines](#) library, with the following naming convention:

Drum	Abbreviation
Bass drum, Kick drum	bd
Snare drum	sd
Rimshot	rim
Clap	cp
Closed hi-hat	hh

Crash	cr
Ride	rd
High tom	ht
Medium tom	mt
Low tom	lt



[original von Pbroks13](#)

More percussive sounds:

Source	Abbreviation
Shakers (and maracas, cabasas, etc)	sh

Tambourine	tb
Other percussions	perc
Miscellaneous samples	misc
Effects	fx

Furthermore, strudel also loads instrument samples from [VCSL](#) by default.

To see which sample names are available, open the **sounds** tab in the [REPL](#).

You can also create custom aliases for existing sounds using the **soundAlias** function:

```
1 soundAlias("RolandTR808_bd", "kick")
2 s("kick")
```

Note that only the sample maps (mapping names to URLs) are loaded initially, while the audio samples themselves are not loaded until they are actually played. This behaviour of loading things only when they are needed is also called **lazy loading**. While it saves resources, it can also lead to sounds not being audible the first time they are triggered, because the sound is still loading. [This might be fixed in the future](#)

Sound Banks

If we open the **sounds** tab and then **drum-machines**, we can see that the drum samples are all prefixed with drum machine names: **RolandTR808_bd**, **RolandTR808_sd**, **RolandTR808_hh** etc..

We *could* use them like this:

```
1 s("RolandTR808_bd RolandTR808_sd,RolandTR808_hh*16")
```

```
1 s("bd sd,hh*16").bank("RolandTR808")
```

You could even pattern the bank to switch between different drum machines:

```
1 s("bd sd,hh*16").bank("<RolandTR808 RolandTR909>")
```

Behind the scenes, **bank** will just prepend the drum machine name to the sample name with **_** to get the full name. This of course only works because the name after **_** (**bd**, **sd** etc..) is standardized. Also note that some banks won't have samples for all sounds!

Selecting Sounds

If we open the **sounds** tab again, followed by tab **drum machines**, there is also a number behind each name, indicating how many individual samples are available. For example **RolandTR909_hh(4)** means there are 4 samples of a TR909 hihat available. By default, **s** will play the first sample, but we can select the other ones using **n**, starting from 0:

```
1 s("hh*8").bank("RolandTR909").n("0 1 2 3")
```

Numbers that are too high will just wrap around to the beginning

```
1 s("hh*8").bank("RolandTR909").n("0 1 2 3 4 5 6 7")
```

Here, 0-3 will play the same sounds as 4-7, because **RolandTR909_hh** only has 4 sounds.

Selecting sounds also works inside the mini notation, using **:** like this:

```
1 s("hh*8").bank("RolandTR909").n("0 1 2 3 4 5 6 7")
```

Loading Custom Samples

You can load a non-standard sample map using the `samples` function.

Loading samples from file URLs

In this example we assign names `bassdrum`, `hihat` and `snaredrum` to specific audio files on a server:

```
1 samples({
2   bassdrum: 'bd/BT0AADA.wav',
3   hihat: 'hh27/000_hh27closedhh.wav',
4   snaredrum: ['sd/rytm-01-classic.wav', 'sd/rytm-00-hard.wav'],
5 }, 'https://raw.githubusercontent.com/tidalcycles/Dirt-Samples
6
7 s("bassdrum snaredrum:0 bassdrum snaredrum:1, hihat*16")
```

You can freely choose any combination of letters for each sample name. It is even possible to override the default sounds. The names you pick will be made available in the `s` function. Make sure that the URL and each sample path form a correct URL!

In the above example, `bassdrum` will load:

```
https://raw.githubusercontent.com/tidalcycles/Dirt-Samples/master/bd/BT0AADA.wav
|-----base path -----|---sample path-|
```

Note that we can either load a single file, like for `bassdrum` and `hihat`, or a list of files like for `snaredrum`! As soon as you run the code, your chosen sample names will be listed in `sounds`
-> `user`.

The above way to load samples might be tedious to write out / copy paste each time you write a new pattern. To avoid that, you can simply pass a URL to a **strudel.json** file somewhere on the internet:

```
1 samples('https://raw.githubusercontent.com/tidalcycles/Dirt-Sa
2 s("bd sd bd sd,hh*16")
```

The file is expected to define a sample map using JSON, in the same format as described above. Additionally, the base path can be defined with the **_base** key. The last section could be written as:

```
{
  "_base": "https://raw.githubusercontent.com/tidalcycles/Dirt-Samples/master/",
  "bassdrum": "bd/BT0AADA.wav",
  "snaredrum": "sd/rytm-01-classic.wav",
  "hihat": "hh27/000_hh27closedhh.wav"
}
```

Please note that browsers will often cache **strudel.json** on first load, and keep using the cached version even if the original has been updated. If this bites you (for example while developing a new sample pack), you can force the browser to download a new copy by i.e. changing capitalization of one character in the URL, or adding a URL attribute, such as:

```
samples('https://raw.githubusercontent.com/tidalcycles/Dirt-Samples/master/strudel
```

that gets ignored by GitHub (but changes the URL, forcing the browser to reload every time we increase the version number).

It is also possible, of course, to just remove it from cache (deleting cache in browser Privacy settings, or from the dev console if you're technically minded, or by using a cache deleting extension)

Generating strudel.json

You can use [@strudel/sampler](#) to generate a strudel.json file for you, by running:

```
npx --yes @strudel/sampler --json > strudel.json
```

See other uses of strudel/sampler further below, under “From Disk via @strudel/sampler”.

Github Shortcut

Because loading samples from github is common, there is a shortcut:

```
1 samples('github:tidalcycles/dirt-samples')
2 s("bd sd bd sd, hh*16")
```

The format is `samples('github:<user>/<repo>/<branch>')`. If you omit **branch** (like above), the **main** branch will be used. It assumes a `strudel.json` file to be present at the root of the repository:

```
https://raw.githubusercontent.com/<user>/<repo>/<branch>/strudel.json
```

From Disk via “Import Sounds Folder”

If you don’t want to upload your samples to the internet, you can also load them from your local disk. Go to the **sounds** tab in the REPL and open the **import-sounds** tab below the search bar. Press the “import sounds folder” button and select a folder that contains audio files. The folder you select can also contain subfolders with audio files. Example:

└─ samples


```
| └─ swooplong.wav
|   └─ swooptight.wav
└─ smash
    └─ smashhigh.wav
    └─ smashlow.wav
    └─ smashmiddle.wav
```

In the above example the folder **samples** contains 2 subfolders **swoop** and **smash**, which contain audio files. If you select that **samples** folder, the **user** tab (next to the **import-sounds** tab) will then contain 2 new sounds: **swoop(3)** **smash(3)** The individual samples can be played normally like `s("swoop:0 swoop:1 smash:2")`. The samples within each sound use zero-based indexing in alphabetical order.

From Disk via @strudel/sampler

Instead of loading your samples into your browser with the "import sounds folder" button, you can also serve the samples from a local file server. The easiest way to do this is using [@strudel/sampler](#):

```
cd samples
npx @strudel/sampler
```

Then you can load it via:

```
1 samples('http://localhost:5432/');
2
3 n("<0 1 2>").s("swoop smash")
```

The handy thing about **@strudel/sampler** is that it auto-generates the **strudel.json** file based on your folder structure. You can see what it generated by going to **http://localhost:5432** with your browser.

Specifying Pitch

To make sure your samples are in tune when playing them with **note**, you can specify a base pitch like this:

```
1 samples({
2   'gtr': 'gtr/0001_cleanC.wav',
3   'moog': { 'g3': 'moog/005_Mighty%20Moog%20G3.wav' },
4 }, 'github:tidalcycles/dirt-samples');
5 note("g3 [bb3 c4] <g4 f4 eb4 f3>@2").s("gtr,moog").clip(1)
6 .gain(.5)
```

We can also declare different samples for different regions of the keyboard:

```
1 setcpm(60)
2 samples({
3   'moog': {
4     'g2': 'moog/004_Mighty%20Moog%20G2.wav',
5     'g3': 'moog/005_Mighty%20Moog%20G3.wav',
6     'g4': 'moog/006_Mighty%20Moog%20G4.wav',
7   }, 'github:tidalcycles/dirt-samples')
8
9 note("g2!2 <bb2 c3>!2, <c4@3 [<eb4 bb3> g4 f4]>")
10 .s('moog').clip(1)
11 .gain(.5)
```

The sampler will always pick the closest matching sample for the current note!

Note that this notation for pitched sounds also works inside a **strudel.json** file.

If you don't want to select samples by hand, there is also the wonderful tool called [shabda](#). With it, you can enter any sample name(s) to query from [freesound.org](#). Example:

```
1 samples('shabda:bass:4,hihat:4,rimshot:2')
2
3 $: n("0 1 2 3 0 1 2 3").s('bass')
4 $: n("0 1*2 2 3*2").s('hihat').clip(1)
5 $: n("~ 0 ~ 1 ~ 0 0 1").s('rimshot')
```

You can also generate artificial voice samples with any text, in multiple languages. Note that the language code and the gender parameters are optional and default to **en-GB** and **f**

```
1 samples('shabda/speech:the_drum,forever')
2 samples('shabda/speech/fr-FR/m:magnifique')
3
4 $: s("the_drum*2").chop(16).speed(rand.range(0.85,1.1))
5 $: s("forever magnifique").slow(4).late(0.125)
```

Sampler Effects

Sampler effects are functions that can be used to change the behaviour of sample playback.

begin

A pattern of numbers from 0 to 1. Skips the beginning of each sample, e.g. **0.25** to cut off the first quarter from each sample.

- amount (number|Pattern): between 0 and 1, where 1 is the length of the sample

```
1 samples({ name: 'rave/ΔRFIREADY wav' }, 'σιthub•tidalcycles/di
```

end

The same as `.begin`, but cuts off the end off each sample.

- `length (number|Pattern)`: 1 = whole sample, .5 = half sample, .25 = quarter sample etc..

```
1 s("bd*2,oh*4").end("<.1 .2 .5 1>").fast(2)
```

loop

Loops the sample. Note that the tempo of the loop is not synced with the cycle tempo. To change the loop region, use `loopBegin` / `loopEnd`.

- `on (number|Pattern)`: If 1, the sample is looped

```
1 s("casio").loop(1)
```

loopBegin

Synonyms: **loopb**

Begin to loop at a specific point in the sample (inbetween **begin** and **end**). Note that the loop point must be inbetween **begin** and **end**, and before **loopEnd**! Note: Samples starting with `wt_` will automatically loop! (`wt` = wavetable)

- `time (number|Pattern)`: between 0 and 1, where 1 is the length of the sample

```
1 s("space").loop(1)
2 .loopBegin("<0 .125 .25>")._scope()
```

Synonyms: **loope**

End the looping section at a specific point in the sample (inbetween **begin** and **end**). Note that the loop point must be inbetween **begin** and **end**, and after **loopBegin**!

- **time** (number|Pattern): between 0 and 1, where 1 is the length of the sample

```
1 s("space").loop(1)
2 .loopEnd("<1 .75 .5 .25>")._scope()
```

cut

In the style of classic drum-machines, **cut** will stop a playing sample as soon as another samples with in same cutgroup is to be played. An example would be an open hi-hat followed by a closed one, essentially muting the open.

- **group** (number|Pattern): cut group number

```
1 s("[oh hh]*4").cut(1)
```

clip

Synonyms: **legato**

Multiplies the duration with the given number. Also cuts samples off at the end if they exceed the duration.

- **factor** (number|Pattern): = 0

```
1 note("c a f e").s("piano").clip("<.5 1 2>")
```

Makes the sample fit the given number of cycles by changing the speed.

```
1 samples({ rhodes: 'https://cdn.freesound.org/previews/132/1320
2 s("rhodes").loopAt(2)
```

fit

Makes the sample fit its event duration. Good for rhythmical loops like drum breaks. Similar to `loopAt`.

```
1 samples({ rhodes: 'https://cdn.freesound.org/previews/132/1320
2 s("rhodes/2").fit()
```

chop

Cuts each sample into the given number of parts, allowing you to explore a technique known as 'granular synthesis'. It turns a pattern of samples into a pattern of parts of samples.

```
1 samples({ rhodes: 'https://cdn.freesound.org/previews/132/1320
2 s("rhodes")
3 .chop(4)
4 .rev() // reverse order of chops
5 .loopAt(2) // fit sample into 2 cycles
```

striate

Cuts each sample into the given number of parts, triggering progressive portions of each

```
1 s("numbers:0 numbers:1 numbers:2").striate(6).slow(3)
```

slice

Chops samples into the given number of slices, triggering those slices with a given pattern of slice numbers. Instead of a number, it also accepts a list of numbers from 0 to 1 to slice at specific points.

```
1 samples('github:tidalcycles/dirt-samples')
2 s("breaks165").slice(8, "0 1 <2 2*2> 3 [4 0] 5 6 7".every(3, r
```

```
1 samples('github:tidalcycles/dirt-samples')
2 s("breaks125").fit().slice([0,.25,.5,.75], "0 1 1 <2 3>")
```

splice

Works the same as slice, but changes the playback speed of each slice to match the duration of its step.

```
1 samples('github:tidalcycles/dirt-samples')
2 s("breaks165")
3 .splice(8, "0 1 [2 3 0]@2 3 0@2 7")
```

scrub

Allows you to scrub an audio file like a tape loop by passing values that represents the position in the audio file in the optional array syntax ex: "0.5:2", the second value controls the speed of playback

```
1 samples('github:switchangel/pad')
2 s("swpad:0").scrub("{0.1!2 .25@3 0.7!2 <0.8:1.5>}%8")
```

```
1 samples('github:yaxu/clean-breaks/main');
2 s("amen/4").fit().scrub("{0@3 0@2 4@3}%8".div(16))
```

speed

Changes the speed of sample playback, i.e. a cheap way of changing pitch.

- `speed (number|Pattern)`: inf to inf, negative numbers play the sample backwards.

```
1 s("bd*6").speed("1 2 4 1 -2 -4")
```

```
1 speed("1 1.5*2 [2 1.1]").s("piano").clip(1)
```

After samples, let's see what Synths afford us.